Scaling Laws for Neural Language Models

Krishna Saluja

Introduction

This is a replication of research paper "Scaling Laws for Neural Language Models" (Kaplan et al., 2020), in which we are validating the core hypothesis of scaling theory: that language modelling cross-entropy loss follows predictable power law relations with respect to model parameters (N) and dataset size (D). Divided into two levels:

1. Level 1: Architectural Implementation
2. Level 2: Scaling Law Validation

Level 1: Making a Decoder-Only Transformer from Scratch

Mathematical Formalism/Theory Used

1. Positional Embeddings

To encode the order of tokens in a sequence, positional information is added to the token embeddings.

Learnable Positional Embeddings

$$P = \text{Embedding}(pos)$$

where:

- P = positional embedding matrix
- pos = position index $(0, 1, \ldots, T-1)$
- $P \in \mathbb{R}^{T \times d_{model}}$

Fixed Sinusoidal Positional Embeddings

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2}{d_{model}}}}\right)$$

where:

- $PE(pos, j)$ = positional encoding at position pos and embedding dimension j
- pos = token position in the sequence
- i = embedding dimension index

- d_{model} = embedding dimension

The final input representation is:

$$X = E + P$$

where:

- X = input to the Transformer
- E = token embeddings
- P = positional embeddings

2. Query, Key and Value Projections

$$Q = XW_Q$$
$$K = XW_K$$
$$V = XW_V$$

where:

- X = input embedding matrix
- Q = query matrix
- K = key matrix
- V = value matrix
- W_Q, W_K, W_V = learnable projection matrices
- $W_Q, W_K, W_V \in \mathbb{R}^{d_{model} \times d_k}$
- d_k = dimension of each attention head

3. Scaled Dot-Product Attention

The raw attention scores are computed as:

$$S = \frac{QK^T}{\sqrt{d_k}}$$

where:

- S = scaled attention score matrix
- Q = query matrix
- K^T = transpose of the key matrix
- d_k = dimension of each attention head

To preserve the autoregressive property, a causal mask is applied:

$$S = \frac{QK^T}{\sqrt{d_k}} + M$$

where:

$$M_{ij} = \begin{cases} 0, & i \geq j \\ -\infty, & i < j \end{cases}$$

and:

- M = causal mask matrix
- i = query position
- j = key position

The normalized attention weights are then obtained using the Softmax function:

$$A = \text{softmax}(S)$$

or equivalently,

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{k=1}^{T} e^{S_{ik}}}$$

where:

- A = attention weight matrix
- A_{ij} = attention weight assigned by token i to token j
- S_{ij} = masked attention score between tokens i and j
- T = sequence length

Finally, the output of a single attention head is computed as:

$$O = AV$$

where:

- O = attention output
- A = attention weight matrix
- V = value matrix

4. Multi-Head Attention

$$\text{MHA}(X) = \text{Concat}(O_1, O_2, \ldots, O_h)W_O$$

where:

- O_i = output of the i^{th} attention head
- h = number of attention heads
- W_O = output projection matrix
- X = input embedding matrix

5. Position-wise Feed Forward Network

$$FFN(x) = W_2(\text{ReLU}(W_1 x + b_1)) + b_2$$

where:

- x = input vector
- $W_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$
- $W_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$
- b_1, b_2 = bias vectors
- $d_{ff} = 4d_{model}$

6. Layer Normalization

$$LN(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where:

- x = input vector
- μ = mean of the features
- σ^2 = variance of the features
- γ = learnable scaling parameter
- β = learnable shifting parameter
- ϵ = small constant for numerical stability

7. Residual Connections

$$y = x + \text{SubLayer}(x)$$

where:

- x = input to the sublayer
- y = output after residual addition
- SubLayer$(\cdot)$ = attention or feed-forward sublayer

For a Pre-Layer Normalization Transformer block:

$$X = x + \text{MHA}(LN(x))$$

$$X = X + FFN(LN(X))$$

where:

- $LN(\cdot)$ = Layer Normalization
- MHA$(\cdot)$ = Multi-Head Attention
- $FFN(\cdot)$ = Position-wise Feed Forward Network

8. Softmax Output Probabilities

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

where:

- p_i = predicted probability of token i
- z_i = logit corresponding to token i
- V = vocabulary size

9. Cross-Entropy Loss

$$L = -\sum_{i=1}^{V} y_i \log(p_i)$$

where:

- L = cross-entropy loss
- y_i = ground-truth label (one-hot encoded)
- p_i = predicted probability for token i
- V = vocabulary size

10. Autoregressive Language Modeling Objective

$$P(x) = \prod_{t=1}^{T} P(x_t \mid x_{<t})$$

where:

- $P(x)$ = probability of the complete sequence
- x_t = token at position t
- $x_{<t}$ = all preceding tokens
- T = sequence length

Replication Methodology

The model was trained on the Tiny Shakespeare dataset, a character-level corpus consisting of Shakespeare's works. This dataset was selected because it eliminates the need for a separate tokenization stage, as each character is treated as an individual token. Consequently, the vocabulary size is limited to 65 unique characters, simplifying the implementation while retaining sufficient complexity to evaluate the Transformer architecture. Total characters in this document are 1,115,394 tokens.

The model operates at the character level, where each input sequence is represented as a sequence of character indices drawn from the 65-character vocabulary. Training samples are generated by randomly selecting fixed-length context windows from the corpus, with the objective of predicting the next character in an autoregressive manner.

The optimization process employs the AdamW optimizer with a learning rate of 3×10^{-4}. Training is performed using a batch size 64. The model is trained for 3000 steps, and the cross-entropy loss is minimized using backpropagation. Training and validation losses are periodically evaluated on randomly sampled batches to monitor convergence and generalization performance.

Level 2: Replicating the Experiments

Scaling Curves

$$\log(L(N)) = \text{constant} - \alpha_N \log(N)$$

$$\log(L(D)) = \text{constant} - \alpha_D \log(D)$$

Calculated Values:

- $\alpha_N = 0.0351$
- $\alpha_D = 0.0856$
- $\gamma = 0.5982$

Note: Reference plots for $\log(L)$ *vs.* $\log(N)$ *(Best-fit line:* $y = -0.0351x + 0.4121$*) and* $\log(L)$ *vs.* $\log(D)$ *(Best-fit line:* $\log L = -0.0586 \log D + 1.2286$*) were observed to fit these exponents.*

Theoretical Discussion

Scaling Ratio Analysis and Comparison with Kaplan et al.

The scaling experiments performed on the Tiny Shakespeare dataset produced empirical values of $\alpha_N = 0.0351$, $\alpha_D = 0.0856$, and a scaling ratio of $\gamma = 0.5982$. As γ is less than 1, the results indicate that increasing the amount of training data provides greater marginal improvements in model performance than increasing the number of model parameters. This suggests that, for the selected dataset, the model benefits more from additional training tokens than from increased parameter capacity.

Kaplan et al. (2020) reported a scaling ratio of approximately $\gamma \approx 0.8$. Although the value obtained in this work is lower, both results support the same overall trend that scaling the dataset is more beneficial than scaling the model size alone. The smaller value observed in this study indicates that the Tiny Shakespeare dataset is more data-limited than parameter-limited. Since the corpus contains only approximately 1.1 million character tokens with a vocabulary of 65 unique characters, increasing the number of parameters beyond a certain point offers diminishing returns unless more training data is available.

The difference between the empirical results and those reported by Kaplan et al. is expected because the two experimental settings are substantially different. The original work evaluated Transformer models containing millions to billions of parameters trained on internet-scale corpora comprising billions of tokens. In contrast, this replication was conducted using considerably smaller models on a character-level dataset of much smaller scale. Differences in dataset size, vocabulary, model capacity, training duration, and the limited range of parameter counts explored all influence the estimated scaling exponents and contribute to the observed discrepancy.

Despite the quantitative difference, the experimental results are consistent with the qualitative conclusions of Kaplan et al. Both studies demonstrate that increasing the amount of training data yields greater improvements in language modeling performance than increasing model parameters alone. The lower empirical value of $\gamma = 0.5982$ further suggests that, within the scale of the Tiny Shakespeare dataset, model performance is constrained primarily by data availability rather than parameter capacity.